

CardioCare: 심장질환 예측 및 모니터링 시스템

1. 프로젝트 개요

1.1 프로젝트 목적

심혈관 질환은 전 세계적으로 높은 사망률을 보이는 질병 중 하나이다. 본 프로젝트는 환자의 임상 정보를 기반으로 심장질환 발생 여부를 예측하는 머신러닝 모델을 개발하고, 실제 운영 환경을 고려한 실험 추적, 모델 저장, Docker 배포, 데이터 드리프트 모니터링 기능까지 포함하는 End-to-End 머신러닝 시스템 구축을 목표로 한다.

1.2 프로젝트 목표

본 프로젝트는 다음 목표를 달성하도록 설계하였다.

- 심장질환 예측 모델 개발
- 데이터 전처리 자동화
- 다양한 머신러닝 모델 비교
- MLflow 기반 실험 관리
- Docker 기반 재현 가능한 실행 환경 제공
- 데이터 드리프트 탐지 및 성능 저하 모니터링
- 단위 테스트를 통한 코드 검증

2. 데이터셋 소개

2.1 사용 데이터셋

본 프로젝트는 UCI Machine Learning Repository의 Heart Disease Dataset을 사용하였다.

사용 데이터:

- Cleveland
- Hungarian
- Switzerland
- VA Long Beach

총 4개의 데이터셋을 통합하여 하나의 학습 데이터셋으로 구성하였다.

2.2 주요 변수

변수명	설명
age	나이
sex	성별
cp	흉통 유형
trestbps	안정시 혈압
chol	혈중 콜레스테롤
fbs	공복 혈당
restecg	심전도 결과
thalach	최대 심박수
exang	운동성 협심증
oldpeak	ST 변화
slope	ST 기울기
ca	주요 혈관 수
thal	지중해빈혈 유형

목표 변수는 심장질환 여부이며 다음 기준으로 이진 분류를 수행하였다.

- num & 0 → 질환 있음(1)
- num = 0 → 질환 없음(0)

3. 데이터 전처리

3.1 결측치 처리

원본 데이터에는 '?' 형태의 결측값이 포함되어 있었다.

전처리 과정에서는 다음 작업을 수행하였다.

- 결측값을 NaN으로 변환
- 수치형 변수 변환
- 이상치 탐색
- 임상적 범위 검증

3.2 이상치 분석

IQR(Interquartile Range) 기반 이상치 탐지를 수행하였다.

생성 파일:

- iqr_outlier_summary.csv

이를 통해 비정상적으로 큰 혈압, 콜레스테롤 등의 데이터를 확인하였다.

3.3 데이터 분할

데이터는 다음 비율로 분할하였다.

- 학습 데이터: 80%
- 테스트 데이터: 20%

Stratified Split을 사용하여 클래스 비율을 유지하였다.

4. 모델 개발 및 성능 비교

4.1 사용 모델

총 5개의 모델을 비교하였다.

1. Logistic Regression
2. Support Vector Machine (SVC)
3. Random Forest
4. K-Nearest Neighbors
5. Tuned Random Forest (GridSearchCV)

4.2 평가 지표

다음 지표를 사용하였다.

- Balanced Accuracy
- Precision
- Recall
- F1 Score

심장질환 예측에서는 실제 환자를 놓치지 않는 것이 중요하므로 Recall을 우선적으로 고려하였다.

4.3 최종 결과

실험 결과 SVC 모델이 가장 우수한 성능을 보였다.

Metric	Score
Balanced Accuracy	0.8435
Precision	0.8763
Recall	0.8333
F1 Score	0.8543

최종 선택 모델:

Support Vector Classifier (SVC)

4.4 모델 저장

최종 모델은 다음 위치에 저장된다.

models/model.joblib

모델 메타데이터는 다음 파일에 저장된다.

models/model_metadata.json

5. 실험 관리 및 재현성

5.1 MLflow 활용

MLflow를 이용하여 실험을 추적하였다.

기록 항목:

- 모델 종류
- 하이퍼파라미터
- Cross Validation 결과
- Test 성능
- Confusion Matrix
- Feature Selection 결과

생성되는 실험:

- logistic_regression
- svc
- random_forest
- knn
- random_forest_tuned

이를 통해 실험 결과를 체계적으로 비교할 수 있다.

5.2 단위 테스트

프로젝트의 안정성을 위해 unittest를 작성하였다.

실행 결과:

Ran 4 tests in 0.290s

OK

모든 테스트를 통과하였다.

6. 배포 및 모니터링

6.1 Docker 기반 실행

Docker를 이용하여 운영체제와 관계없이 동일한 환경에서 실행 가능하도록 구성하였다.

주요 장점:

- 환경 의존성 제거
- 재현성 확보
- 간편한 배포

사용 예시:

```
docker build -t cardiocare:1.0 .
```

```
docker run --rm cardiocare:1.0
```

6.2 데이터 드리프트 탐지

실제 운영 환경에서는 데이터 분포가 변화할 수 있다.

이를 탐지하기 위해 Kolmogorov-Smirnov Test 기반 드리프트 분석을 수행하였다.

실행 결과:

Feature	Drift
trestbps	True
chol	True
oldpeak	True

6.3 성능 저하 분석

드리프트 발생 시 모델 성능 변화를 측정하였다.

Dataset	Balanced Accuracy
Original Test	0.8435
Drifted Test	0.8191

드리프트 발생 후 성능이 감소하는 것을 확인하였다.

이는 실제 운영 환경에서 지속적인 모니터링과 재학습이 필요함을 보여준다.

7. 결론

본 프로젝트에서는 심장질환 예측을 위한 End-to-End 머신러닝 시스템을 구축하였다.

주요 성과는 다음과 같다.

- UCI Heart Disease 데이터 통합
- 자동 전처리 파이프라인 구축
- 5개 모델 비교 실험 수행
- SVC 모델 최종 선정
- MLflow 기반 실험 추적
- Docker 기반 배포 환경 제공
- 데이터 드리프트 탐지 기능 구현
- 단위 테스트를 통한 품질 검증

최종 모델은 Balanced Accuracy 84.35%, Recall 83.33%를 달성하였으며, 운영 환경을 고려한 모니터링 기능까지 포함하여 실제 머신러닝 서비스 구조를 학습할 수 있는 프로젝트로 완성하였다.